



Design Document

(DOC No. POC TBD)

>> **Himax WE2 POC for Microchip**

WiseEye™ AI Sensing Solution

Preliminary version 01 Dec 2025

IMPORTANT NOTICE:

This design document is preliminary and certain content may not be fully validated.

>>HX6538-A (WE2)

WiseEye™ AI Processor



Himax Technologies, Inc.

<http://www.himax.com.tw>

Revision History

Dec 2025

Version	Date	Description of changes
01	2025 / 12 / 17	New setup.
02	2026 / 01 / 21	Add I3C commands

List of Contents

Dec 2025

1.	Introduction	6
2.	Himax WE2 AI Sensing Features	7
3.	Himax Hardware and Platforms	8
4.	WE2 POC System Architecture	9
5.	Configuring and Connecting to WE2 EVB	11
1.1	System Block Diagram of HX6538-A06DFG-EVB	11
1.2	Appearance and dimension of HX6538-A06DFG-EVB	11
1.3	CIS Module	12
1.4	WE2 Boot Selection	12
1.5	Debug Board Connector	12
1.6	Power Measurement Test Points	12
1.7	WE2 IO Interface (JP11)	13
1.8	Connecting External Host to WE2 EVB	14
6.	WE2 I2C Overview	14
4.1	I2C Description	14
4.2	I2C Format	14
4.3	I2C Command Descriptions	16
4.4	I2C Command Examples	16
4.5	I2C Sample Python Script	16
4.5.1	Setting I2C transfer speed to to 400 Kbps	16
4.5.2	Slave address is 0x62	16
4.5.3	Write command to trigger I2C communication without any payload	16
4.5.4	Read command (read 9 bytes)	17
7.	WE2 I3C Overview	19
7.1	I3C Description	19
8.	Security	24
9.	Firmware Update	25

List of Figures

Dec 2025

FIGURE 3.1: HM01B0 CCM DRAWING	8
FIGURE 4.1: MICROCHIP POC WE2 SYSTEM BLOCK DIAGRAM.....	9
FIGURE 4.2: MICROCHIP POC WE2 REFERENCE CIRCUIT	10
FIGURE 5.1: HX6538-A06DFG-EVB (QFN88) BLOCK DIAGRAM.....	11
FIGURE 5.2: HX6538-A06DFG-EVB (QFN88) APPEARANCE AND DIMENSION	11
FIGURE 5.3: HX6538-A06DFG-EVB (QFN88) KEY COMPONENTS.....	12
FIGURE 5.4: HX6538-A06DFG-EVB (QFN88) NORMAL MODE BOOT OPTION.....	12
FIGURE 5.5: HX6538-A06DFG-EVB (QFN88) FLASH MODE BOOT OPTION.....	12
FIGURE 5.6: HX6538-A06DFG-EVB (QFN88) POWER MEASUREMENT TEST POINTS	13
FIGURE 5.7: HX6538-A06DFG-EVB (QFN88) IO INTERFACE (JP11).....	13
FIGURE 5.8: HX6538-A06DFG-EVB (QFN88) JP11 PIN OUT.....	13



List of Tables

Dec 2025

TABLE 2.1: WE2 FEATURE..... 7

TABLE 4.1: WE2 PIN ASSIGNMENT 9

1. Introduction

This document details the collaborative effort between Microchip Corporation and Himax Technologies to develop a low power contextual sensing telemetry Proof of Concept (POC) platform. The platform will be built upon the Himax WiseEye2 (WE2) AI sensing solution, which will be designed and developed by Himax in accordance with Microchip's specifications and the mutually agreed statement of work.

The Himax solution will feature the ultra-low power HX6538 AI NPU processor, the HM01B0 CMOS Image Sensor (CIS), and the WE2 machine learning algorithm. It will also include the necessary firmware and drivers to enable the following Microchip-defined features:

- Detection of whether the user is engaged or unengaged from the sensor
- Measurement of the user's distance and angle from the sensor
- Recognition of user movement, including moving away from or towards the sensor
- Identification of whether the user is sitting or standing
- Measurement of distance and angle for non-users, supporting up to five people

Microchip will be responsible for defining the POC system, integrating the components, validating the solution, and demonstrating its capabilities.

2. Himax WE2 AI Sensing Features

- Laptop application – always on context awareness of user and onlookers
- AI sensor (WE2 + HM01B0) placed on Notebook bezel (hinge-up)
- AI sensor Target features listed below

User environment: User workspace (e.g. desk in office) or conference room (>8ft)

Subject	Model / Training	Telemetry Feature	Telemetry Metadata Output via I2C or I3C
User	FD or BD (select based on distance)	Model	Confidence, x, y, w, h, ID0
		User distance	Far ($\geq 8\text{ft}$), mid (5 ft to 7ft), near ($\leq 5\text{ ft}$)
		User angle from camera (assume lid = 120°)	Angle
		User move away / move towards	Towards or away
		User sitting or standing	Sit or Stand
	Head pose	Model	Confidence, yaw, roll, pitch
		User engaged / not engaged with laptop device	Engage or Not engaged
Non-User	FD or BD of up to 5 people	Model (up to 5 people)	Confidence, x, y, w, h, ID# [1 – 5]
		People distance	Far ($\geq 8\text{ft}$), mid (5 ft to 7ft), near ($\leq 5\text{ ft}$)
		People angle from camera (assume lid = 120°)	Angle

Table 2.1: WE2 Feature

3. Himax Hardware and Platforms

This section describes the Himax hardware and related documents used in this POC.

- HX6538 NPU Component
 - Part Number: HX6538-A06TDFG
 - Package Type: QFN88
 - Datasheet: HX6538-A_DS_v04_250711 (version at time of this document)
 - Application Note: HX6538-A_AN_preliminary_v04
- HX6538 Evaluation Board
 - Part Number (EVB): WE2_A06TDFG-EVB
 - Part Number (Debug Board): B23-J000-00
 - WE2 Package Type: QFN88
 - Sensor Module: HM01B0 CCM
 - Datasheet: HX6538-A_HWUG(EVB)_preliminary_v02
- HM01B0 CIS Component
 - Part Number: HM01B0-MWA
 - Datasheet: HM01B0_DS_v13
 - Package Information: HM01B0-PI-Package Information-Ver08
 - Application Note: HM01B0_ENG AN_v06
- HM01B0 Sensor Camera Module (includes focused lens, holder, PCB substrate, board connector)
 - Part Number: HM01B0-MWA-00PT873
 - Module dimension: 9mm x 9mm
 - Lens Part Number: PT1033A-131
 - Lens HFOV: 87°

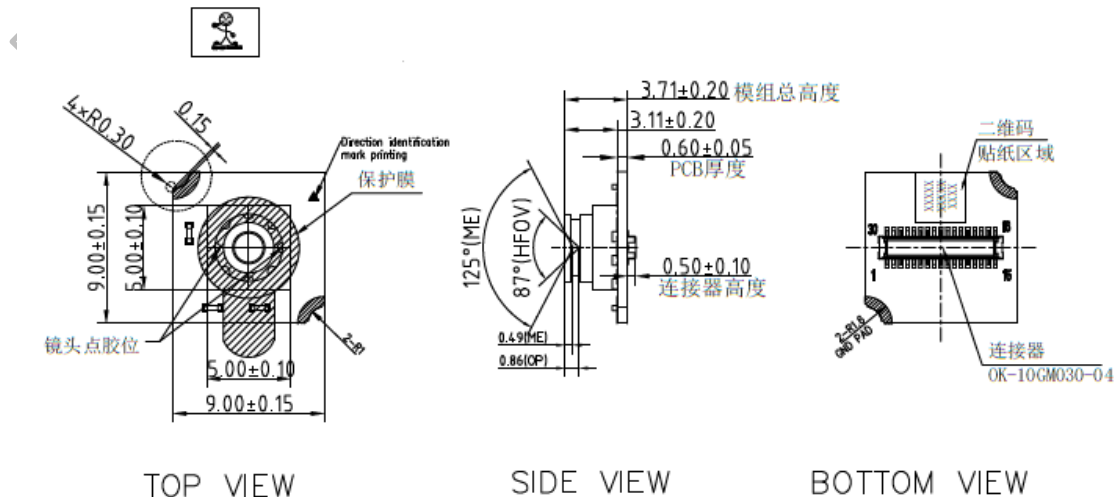


Figure 3.1: HM01B0 CCM Drawing

4. WE2 POC System Architecture

WE2 interfaces to an external host controller using the following predefined HX6538 IO pins. Himax will provide Microchip a firmware image to support the predefined pin functions.

Purpose	WE2 Function	WE2 Pin Name	System Function
MCU Data Interface	I2C or I3C (programmable)	PA2 (pin4) SDA PA3 (pin5) SCL	- Firmware update - AI sensor metadata output interface - Host control interface
MCU Trigger	Wake from DPD / PD	PA0 (pin1) PA1 (pin2)	Wakes HX6538 from Deep Power Down (no retention) or Power Down (retention)
MCU Interrupt	Event Interrupt (optional)	PB0 (pin11)	Programmable event interrupt to host MCU
UART Tx	UART Tx	PB1 (pin12)	UART for WE2 debug
MCU Control (IO)	FPS_select (optional)	PB2 (pin31)	Throttle WE2 event detection frame rate by GPIO control from host MCU
	(To be defined)	PB3 – PB11	Programmable GPIO expansion pins

Table 4.1: WE2 Pin Assignment

WE2 + HM01B0 system architecture and application circuits are shown in the figures below. The schematic (HX6538(QFN88) + HM01B0(CCM)_V10_20250919.dsn) have been released to Microchip.

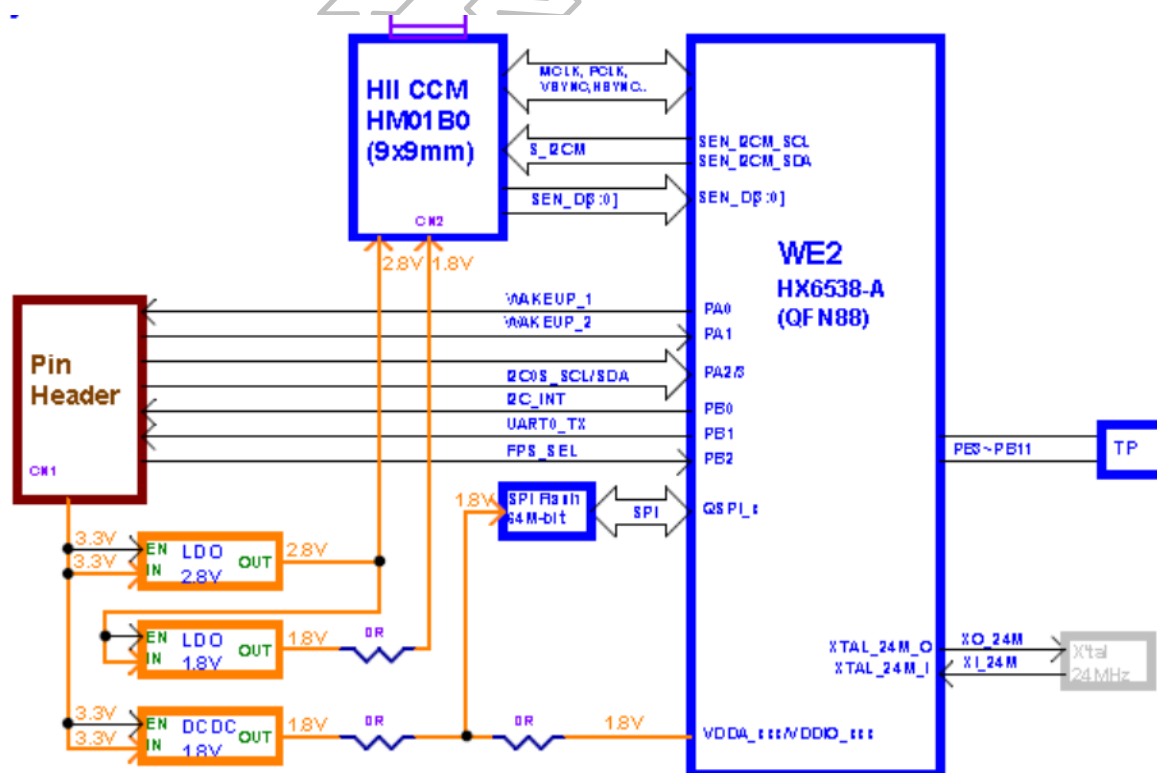


Figure 4.1: Microchip POC WE2 System Block Diagram

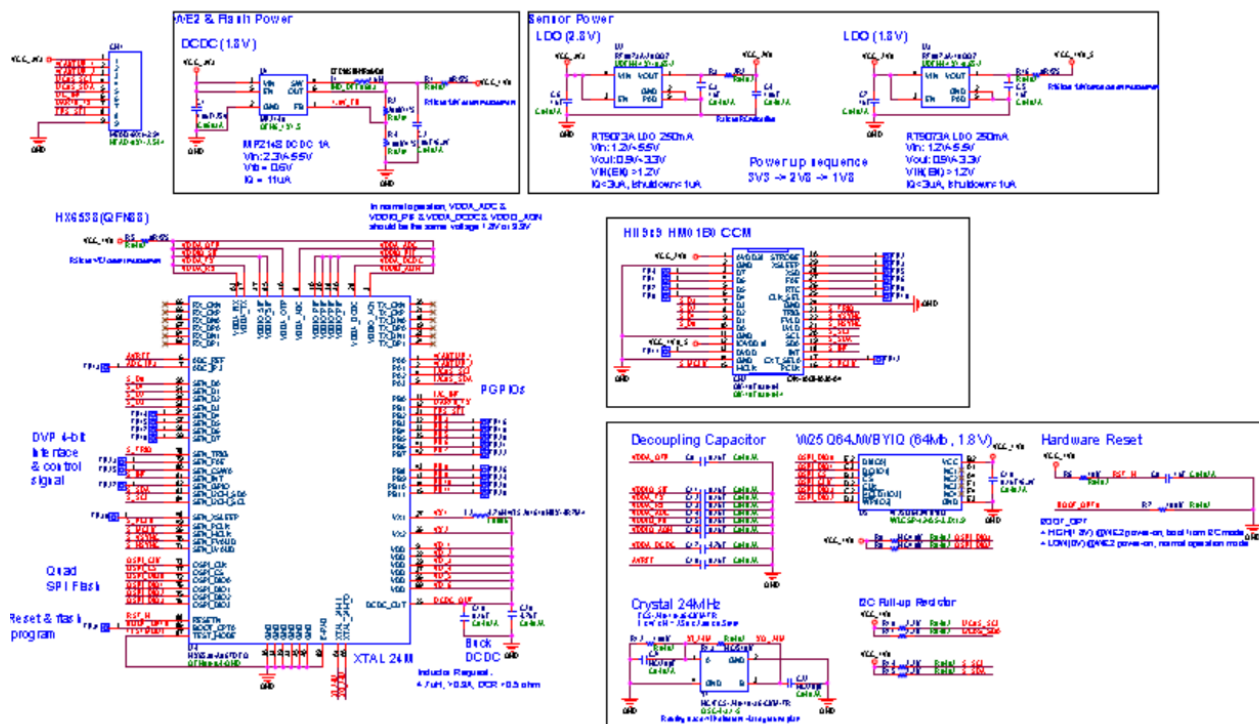


Figure 4.2: Microchip POC WE2 Reference Circuit

5. Configuring and Connecting to WE2 EVB

This section provides relevant project information using WE2 EVB to support the system architecture in the previous section. Please reference Himax HX6538-A_HWUG(EVB)_preliminary_v02 document for all the information.

5.1 System Block Diagram of HX6538-A06DFG-EVB

Please reference Himax HX6538-A HWUG(EVB) preliminary v02 document for the complete information.

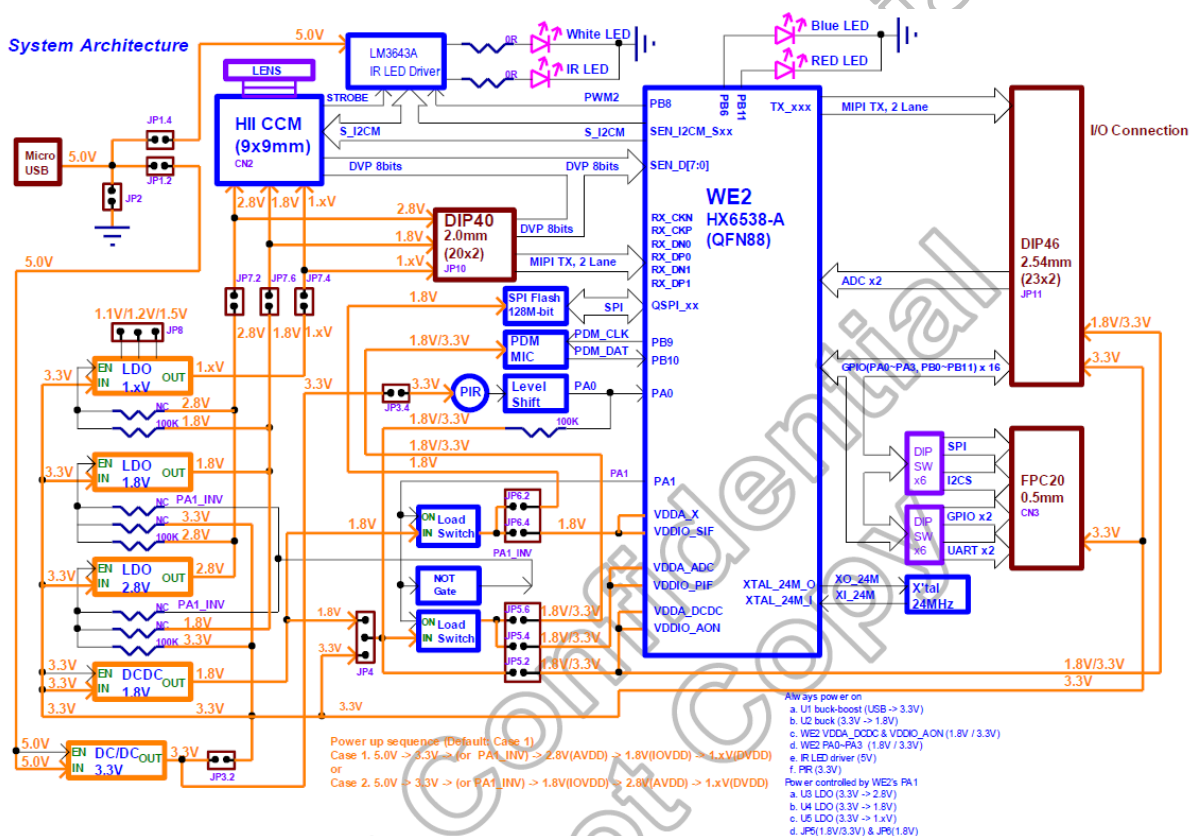


Figure 5.1: HX6538-A06DFG-EVB (QFN88) Block Diagram

5.2 Appearance and dimension of HX6538-A06DFG-EVB

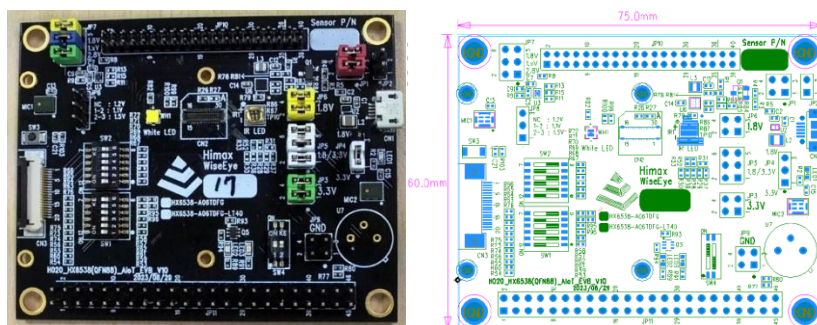


Figure 5.2: HX6538-A06DFG-EVB (QFN88) appearance and dimension

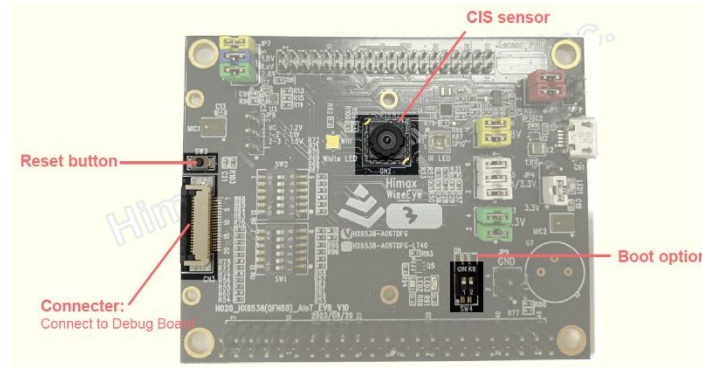


Figure 5.3: HX6538-A06DFG-EVB (QFN88) key components

5.3 CIS Module

Removable 9mm x 9mm sensor camera module (HM01B0 CIS + focused lens on PCB substrate with a board to board connector) that mounts on the EVB.

5.4 WE2 Boot Selection

WE2 QFN88 supports boot option from external QSPI flash memory or I2C. The boot option is configured using on-board switch (SW4) as shown in the figures below.

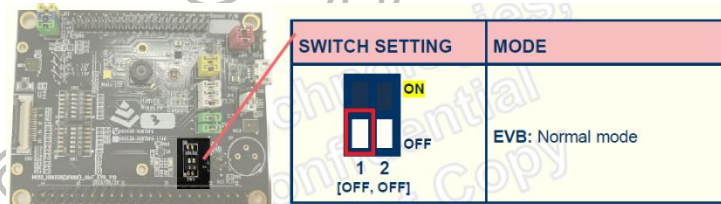


Figure 5.4: HX6538-A06DFG-EVB (QFN88) normal mode boot option

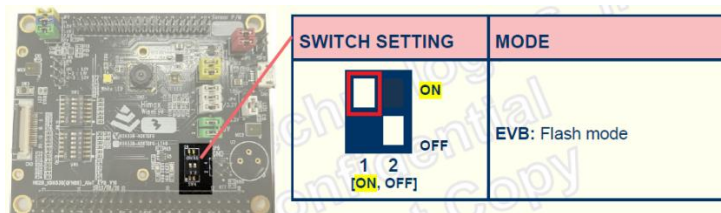


Figure 5.5: HX6538-A06DFG-EVB (QFN88) Flash mode boot option

5.5 Debug Board Connector

A 20-pin PFC cable connects WE2 EVB to debug board. The debug board incorporates a bridge chip which connects WE2 I2C, UART and SPI to Windows PC host over USB2.

5.6 Power Measurement Test Points

Test points JP5.2, JP5.4, JP6.2, and JP6.4 support isolated current measurements for HX6538.

- JP5.2: Core
- JP5.4: Peripheral Interface (PIF)
- JP6.2: External Flash

- JP6.4: Sensor Interface (SIF)

Note: JP5.6 is for external PDM MIC. Unless it is used, it is not required to be measured.

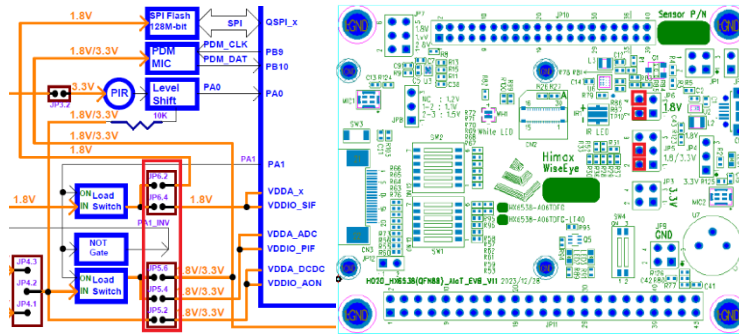


Figure 5.6: HX6538-A06DFG-EVB (QFN88) Power Measurement Test Points

5.7 WE2 IO Interface (JP11)

JP11 is located near lower edge of EVB, and provides I/O interface of WE2.

To connect WE2 EVB to an external MCU (e.g. I2C over PA2 / PA3), connect the IO using the assigned pins on JP11 and disconnect debug board. The EVB can operate without debug board since it can be powered by USB port located on upper-right side.

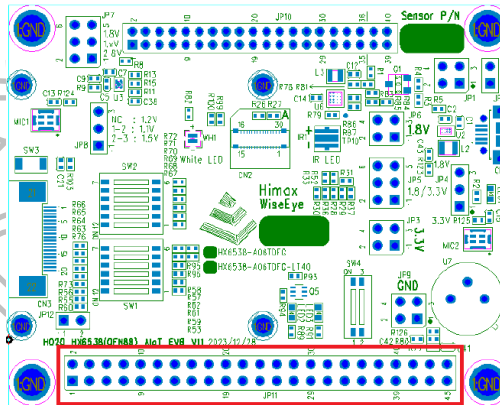


Figure 5.7: HX6538-A06DFG-EVB (QFN88) IO Interface (JP11)

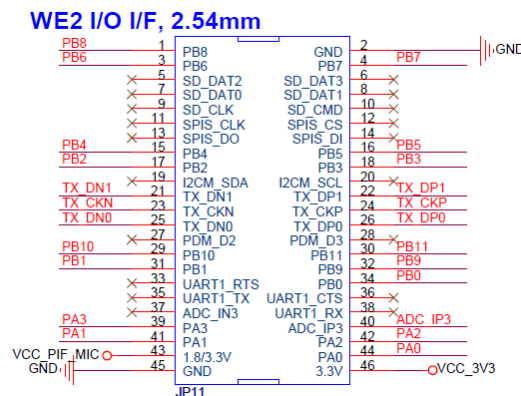


Figure 5.8: HX6538-A06DFG-EVB (QFN88) JP11 Pin Out

5.8 Connecting External Host to WE2 EVB

The predefined I2C and GPIO can be connected to an external host processor from WE2 EVB JP11. It is recommended to disconnect the debug board. The EVB can operate without debug board. It can be powered by USB port located on upper-right side.

6. WE2 I2C Overview

6.1 I2C Description

HX6358 supports both master and slave operation. For Microchip POC, PA2 / PA3 are configured for I2C slave.

- Slave ID: 0x62 (7bits)
- Programmable 7-bit or 10-bit addressing
 - OTA loader uses 7-bit addressing mode
- Support clock stretching
- Transmission modes
 - Standard mode (up to 100Kb/s)
 - Fast mode (up to 400Kb/s) - default
 - Fast mode plus (up to 1000Kb/s)

6.2 I2C Format

• Packet format description

Host Device(Master) sends write command	slv_addr (7bits)	R/W 1bit	Feature	cmd_type	Length	Data payload (N=Length) (MAX:256)				checksum
						cmd_param 1	cmd_param 2	...	cmd_param N	
I2C Format	SlaveID	0	wbuf[0]	wbuf[1]	wbuf[3:2]	wbuf[4]	wbuf[5]	...	wbuf[N+3]	wbuf[N+4] wbuf[N+5]

[Field Description]

slv_addr	i2c slave address (default: 0x62)
R/W	1'b0 - for write command
Feature	indicate the supported feature in WE2
cmd_type	indicate the supported cmd for specific feature
Length	total byte count in data payload
cmd_param N	parameter for specific cmd_type
checksum	the checksum value to verify the serial data is correct or not during transition on i2c bus

Host Device(Master) sends read command	slv_addr (7bits)	R/W 1bit	Feature	cmd_type	Length	Data payload (N=Length) (MAX:256)				checksum
						rdata 1	rdata 2	...	rdata N	
I2C Format	SlaveID	1	rbuf[0]	rbuf[1]	rbuf[3:2]	rbuf[4]	rbuf[5]	...	rbuf[N+3]	rbuf[N+4] rbuf[N+5]

[Field Description]

slv_addr	i2c slave address (default: 0x62)
R/W	1'b1 - for read command
Feature	the value should be same with the field:Feature of previous write command.
cmd_type	the value should be same with the field:cmd_type of previous write command.
Length	total byte count in data payload. (the length is pre-define by cmd_type or configure by cmd_param from W/R cmd_type)
rdata N	data buffer for master read command
checksum	the checksum value to verify the serial data is correct or not during transition on i2c bus

checksum algorithm : crc16_ccitt

Checksum range = buf[0] ⊕ buf[1] ⊕ buf[2] ⊕ buf[3] ⊕ ... ⊕ buf[N+3]

Checksum calculation starts with byte[0], ends with last byte before checksum bytes. In other words, it starts with Feature, followed by cmd_type, payload length, and ends with the last byte of payload.

Checksum calculation does not include checksum itself.

slv_addr (7bits)	R/W 1bit	Feature	cmd_type	Length	Data payload (N=Length) (MAX:256)				CRC (ccitt_crc16)
					cmd_param 1	cmd_param 2	...	cmd_param N	
SlaveID	0	wbuf[0]	wbuf[1]	wbuf[3:2]	wbuf[4]	wbuf[5]	...	wbuf[N+3]	wbuf[N+5:N+4]

When calculating checksum, algorithm CRC16/MCRF4XX is used, with option of MSB Right (Little

Endian).

Himax Confidential
Do Not Copy

6.3 I2C Command Descriptions

Feature	Feature code	Command type	Command code	Host Property	Description
I2CCOMM_FEATU RE_OTA_OP	0x50	I2CCOMM_CMD_OTA_GET_VER	0x01	W/R	Get OTA loader version
		I2CCOMM_CMD_OTA_READ_CHIP_ID	0x03	W/R	Get WE2 Chip ID
		I2CCOMM_CMD_OTA_READ_STATUS	0x04	W/R	Get WE2 current status
		I2CCOMM_CMD_OTA_REBOOT	0x05	W/O	Set WE2 to reboot itself.
		I2CCOMM_CMD_OTA_BUSY_PIN_SELECT	0x06	W/O	Set WE2 handshake GPIO pin
		I2CCOMM_CMD_OTA_READ_BUSY_PIN	0x07	W/R	Get WE2 handshake GPIO
		I2CCOMM_CMD_OTA_JUMP2UPG(=Intoupg)	0x08	W/O	Set WE2 jump into OTA loader
I2CCOMM_FEATU RE_OTA_UPG	0x51	I2CCOMM_CMD_OTA_START	0x09	W/O	OTA start to initialize OTA loader
		I2CCOMM_CMD_OTA_DATA	0x0A	W/O	Send image files to OTA loader
		I2CCOMM_CMD_OTA_END	0x0B	W/O	Send OTA update end. OTA start to verify image and write image to flash

- Note:
- Host Property
- WE2 is slave, PC(host) is master. W:host write, R:host read.

6.4 I2C Command Examples

Developers can follow the examples below to send I2C command to WE2 OTA loader.

Command type	Host R/W	Send Command Description											
I2CCOMM_CMD_OTA_GET_VER		I2C Address		Header			Data payload (N=Length) (MAX:256)						checksum
		slv_addr (7bits)	R/W 1bit	Feature	cmd_type	Length	cmd_param 1	cmd_param 2	...	cmd_param N			
		SlaveID	0	wbuf[0]	wbuf[1]	wbuf[2]	wbuf[3]	wbuf[4]	wbuf[5]	...	wbuf[N+3]	wbuf[N+4]	wbuf[N+5]
		Write	0x62	0	0x50	0x01	0x0	0x0	...	NULL	...	CRC value	
I2CCOMM_CMD_OTA_READ_CHIP_ID		I2C Address		Header			Data payload (N=Length) (MAX:256)						checksum
		slv_addr (7bits)	R/W 1bit	Feature	cmd_type	Length	cmd_param 1	cmd_param 2	...	cmd_param N			
		SlaveID	0	wbuf[0]	wbuf[1]	wbuf[2]	wbuf[3]	wbuf[4]	wbuf[5]	...	wbuf[N+3]	wbuf[N+4]	wbuf[N+5]
		Write	0x62	0	0x50	0x03	0x0	0x0	...	NULL	...	CRC value	
I2CCOMM_CMD_OTA_READ_STATUS		I2C Address		Header			Data payload (N=Length) (MAX:256)						checksum
		slv_addr (7bits)	R/W 1bit	Feature	cmd_type	Length	cmd_param 1	cmd_param 2	...	cmd_param N			
		SlaveID	0	wbuf[0]	wbuf[1]	wbuf[2]	wbuf[3]	wbuf[4]	wbuf[5]	...	wbuf[N+3]	wbuf[N+4]	wbuf[N+5]
		Read	0x62	1	0x50	0x04	0x0	0x0	...	NULL	...	CRC value	
I2CCOMM_CMD_OTA_READ_STATUS		I2C Address		Header			Data payload (N=Length) (MAX:256)						checksum
		slv_addr (7bits)	R/W 1bit	Feature	cmd_type	Length	cmd_param 1	cmd_param 2	...	cmd_param N			
		SlaveID	0	wbuf[0]	wbuf[1]	wbuf[2]	wbuf[3]	wbuf[4]	wbuf[5]	...	wbuf[N+3]	wbuf[N+4]	wbuf[N+5]
		Read	0x62	1	0x50	0x04	0x0	0x0	...	4 bytes WE2 status value	...	CRC value	

6.5 I2C Sample Python Script

6.5.1 Setting I2C transfer speed to 400 Kbps

```

26 class HmxI2cCmd():
27     def __init__(self):
28         self.i2c = I2cMaster(400)

```

6.5.2 Slave address is 0x62

```

42 # I2C Master Write data to dev_id 0x62.
43 self.i2c.write(0x62, data)

```

6.5.3 Write command to trigger I2C communication without any payload

```

16 FEATURE_2 = bytearray(b'\x82\x00\x00\x00')

```

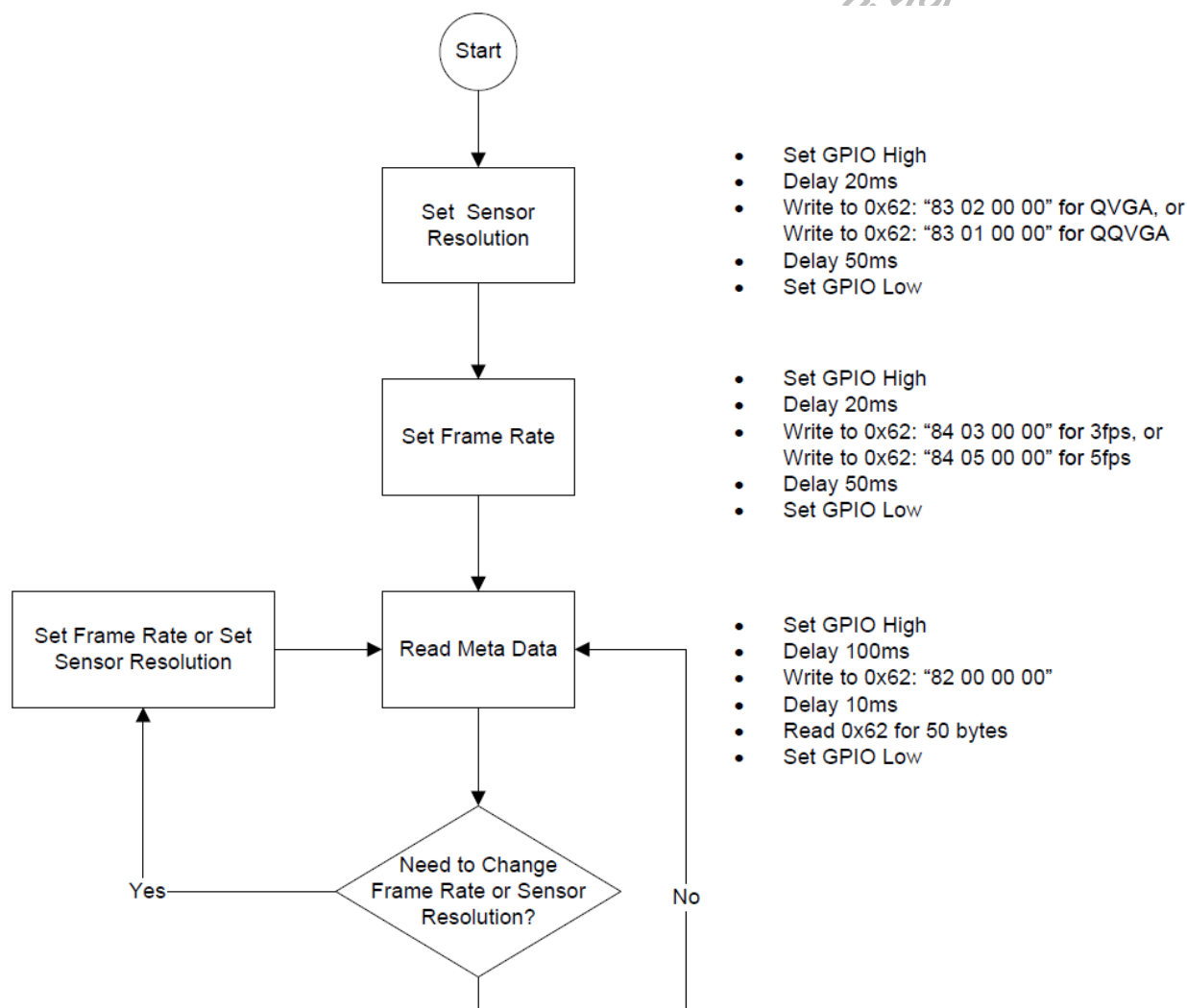
Feature code is 0x82; Command type is 0x00; Length is 0x00, 0x00.

6.5.4 Read command

Below is an example to read 54 bytes (4 header bytes + 50 payload bytes)

```
# I2C Master Read the feedback data from 0x62
sen_reg_read_data = self.i2c.read(0x62, 54)
```

Below is the flow chart on the host. Note: the delay time is recommended only. The ideal values may depend on the actual timing of the host.



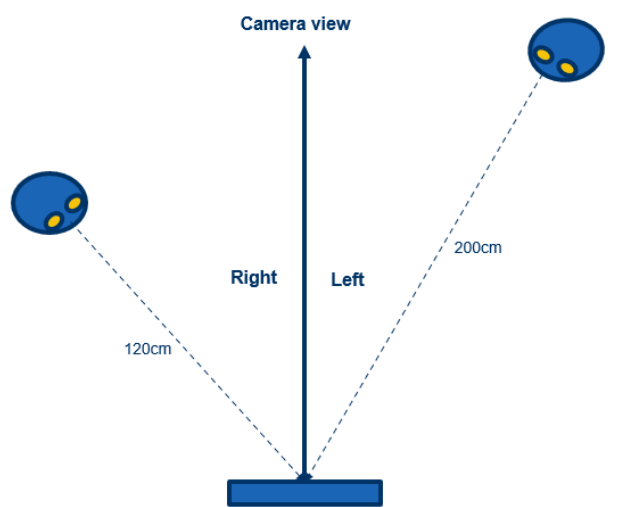
6.6 Metadata format

Metadata can be fetched through I2C by writing the command to slave address 0x62. "82 00 00 00", then read 54 bytes (including 4 bytes of header and 50 bytes of payload). The payload information is defined in the table below.

I2C Byte Index	Metadata Index	Descriptions	Notes
0		Feature ID (0x82)	
1		Command ID (0x00)	
2		Payload length LSB (0x32)	
3		Payload length MSB (0x00)	

4	0	Reserved (0x00)	
5	1	Reserved (0x00)	
6	2	User0 confidence	Range: 0 - 100
7	3	User0 bounding-box x-coordinate (upper-left corner)	Unit: number of pixels
8	4	User0 bounding-box y-coordinate (upper-left corner)	Unit: number of pixels
9	5	User0 bounding-box width	Unit: number of pixels
10	6	User0 bounding-box height	Unit: number of pixels
11	7	User0 distance category	3: Far (≥ 8 ft); 2: mid (5 ft to 7ft); 1: near (≤ 5 ft); 0: N/A
12	8	User0 angle	Unit: degree. Range: -90 to 90. Polarity: positive on left pane; negative on right pane. Illustrated in chart below.
13	9	User0 toward or away	1: move away; 2: move toward; 0: N/A
14	10	User0 sitting or standing	2: standing; 1: sitting; 0: N/A
15	11	User0 yaw	Unit: degree
16	12	User0 pitch	Unit: degree
17	13	User0 roll	Unit: degree
18	14	User0 engaged	1: User in attention; 0: User not in attention
19	15	Non-user1 confidence	Range: 0 - 100
20	16	Non-user1 bounding-box x-coordinate (upper-left corner)	Unit: number of pixels
21	17	Non-user1 bounding-box y-coordinate (upper-left corner)	Unit: number of pixels
22	18	Non-user1 bounding-box width	Unit: number of pixels
23	19	Non-user1 bounding-box height	Unit: number of pixels
24	20	Non-user1 distance category	3: Far (≥ 8 ft); 2: mid (5 ft to 7ft); 1: near (≤ 5 ft); 0: N/A
25	21	Non-user1 angle	positive on left; negative on right
26	22	Non-user2 confidence	Range: 0 - 100
27	23	Non-user2 bounding-box x-coordinate (upper-left corner)	Unit: number of pixels
28	24	Non-user2 bounding-box y-coordinate (upper-left corner)	Unit: number of pixels
29	25	Non-user2 bounding-box width	Unit: number of pixels
30	26	Non-user2 bounding-box height	Unit: number of pixels
31	27	Non-user2 distance category	3: Far (≥ 8 ft); 2: mid (5 ft to 7ft); 1: near (≤ 5 ft); 0: N/A
32	28	Non-user2 angle (positive on left; negative on right)	positive on left; negative on right
33	29	Non-user3 confidence	Range: 0 - 100
34	30	Non-user3 bounding-box x-coordinate (upper-left corner)	Unit: number of pixels
35	31	Non-user3 bounding-box y-coordinate (upper-left corner)	Unit: number of pixels
36	32	Non-user3 bounding-box width	Unit: number of pixels
37	33	Non-user3 bounding-box height	Unit: number of pixels
38	34	Non-user3 distance category	3: Far (≥ 8 ft); 2: mid (5 ft to 7ft); 1: near

			(≤ 5 ft); 0: N/A
39	35	Non-user3 angle (positive on left; negative on right)	positive on left; negative on right
40	36	Non-user4 confidence	Range: 0 - 100
41	37	Non-user4 bounding-box x-coordinate (upper-left corner)	Unit: number of pixels
42	38	Non-user4 bounding-box y-coordinate (upper-left corner)	Unit: number of pixels
43	39	Non-user4 bounding-box width	Unit: number of pixels
44	40	Non-user4 bounding-box height	Unit: number of pixels
45	41	Non-user4 distance category	3: Far (≥ 8ft); 2: mid (5 ft to 7ft); 1: near (≤ 5 ft); 0: N/A
46	42	Non-user4 angle (positive on left; negative on right)	positive on left; negative on right
47	43	Non-user5 confidence	Range: 0 - 100
48	44	Non-user5 bounding-box x-coordinate (upper-left corner)	Unit: number of pixels
49	45	Non-user5 bounding-box y-coordinate (upper-left corner)	Unit: number of pixels
50	46	Non-user5 bounding-box width	Unit: number of pixels
51	47	Non-user5 bounding-box height	Unit: number of pixels
52	48	Non-user5 distance category	3: Far (≥ 8ft); 2: mid (5 ft to 7ft); 1: near (≤ 5 ft); 0: N/A
53	49	Non-user5 angle (positive on left; negative on right)	positive on left; negative on right



7. WE2 I3C Overview

7.1 I3C Description

HX6538 supports slave operation following MIPI I3C v1.0. For Microchip POC, PA2 / PA3 will be configured as I3C slave. The HX6538 I3C supports the following features:

- In Band Interrupt
- Transmission modes
 - Standard mode (up to 100Kb/s)
 - Fast mode (up to 400Kb/s) - default

- Fast mode plus (up to 1000Kb/s)
- Single data rate messaging (up to 12.5Mb/s)
- High data rate messaging (HBR-DDR up to 25Mb/s)

The supported features for I3C Slave and corresponding host commands are listed:

1) Write metadata trigger.

Type	Command	Flag	Payload Length	Payload
0x01	0x13	0x05	0x00	N/A

Expected response messages.

Type	Command	Flag	Payload Length	Payload
0x01	0x13	0x06	0x00	N/A

2) Read metadata.

Type	Command	Flag	Payload Length	Payload
0x01	0x14	0x05	0x00	N/A

Expected response messages.

Type	Command	Flag	Payload Length	Payload
0x01	0x14	0x06	0x32	See design doc

3) Set sensor resolution.

Type	Command	Flag	Payload Length	Payload
0x01	0x15	0x05	0x01	0x01: resolution 162x122 0x02: resolution 324x224

Expected response messages.

Type	Command	Flag	Payload Length	Payload
0x01	0x15	0x06	0x01	0x01: resolution 162x122 0x02: resolution 324x224

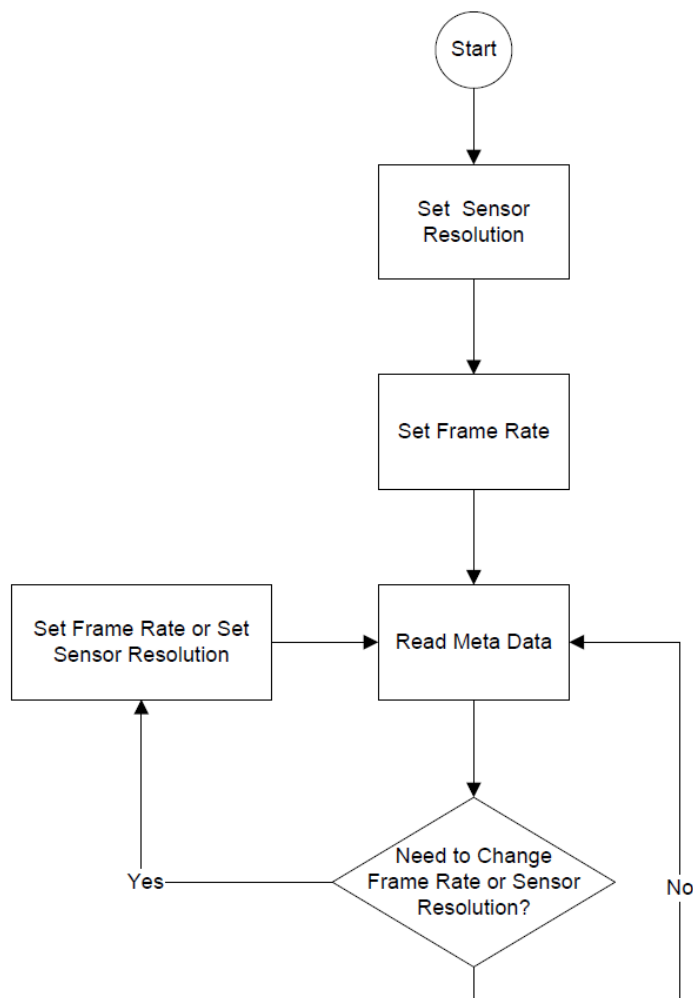
4) Set frame rates.

Type	Command	Flag	Payload Length	Payload
0x01	0x16	0x05	0x01	Frame Per Second (1 – 5)

Expected response messages.

Type	Command	Flag	Payload Length	Payload
0x01	0x16	0x06	0x01	Frame Per Second (1 – 5)

Below is the flow chart on the host. Note: the delay time is recommended only. The ideal values may depend on the actual timing of the host.



- Set GPIO High
- Delay 20ms
- Write to slave: "01 15 05 01 02" for QVGA, or
- Write to slave: "01 15 05 01 01" for QQVGA
- Delay 50ms
- Set GPIO Low

- Set GPIO High
- Delay 20ms
- Write to slave: "01 16 05 01 03" for 3fps, or
- Write to slave: "01 16 05 01 05" for 5fps
- Delay 50ms
- Set GPIO Low

- Set GPIO High
- Delay 100ms
- Write to slave: "01 13 05 00"
- Delay 10ms
- Read from slave: "01 14 05 00"
- Set GPIO Low

For more code examples

You may refer to code snippets below:

Example 1:

```
#include <zephyr/kernel.h> #include <zephyr/drivers/i3c.h> #include <zephyr/logging/log.h>
```

```
LOG_MODULE_REGISTER(main, LOG_LEVEL_INF);
```

```
/* Get the I3C controller device from devicetree */ const struct device *i3c_bus =  
DEVICE_DT_GET(DT_NODELABEL(i3c0));
```

```
void main(void) { if (!device_is_ready(i3c_bus)) { LOG_ERR("I3C bus device not ready");  
return; }
```

```
/* 1. Reset the bus to clear old Dynamic Addresses */  
struct i3c_ccc_payload reset_payload = {0};  
if (i3c_ccc_do_rstdaa(i3c_bus) != 0) {  
    LOG_ERR("Failed to broadcast RSTDAA");  
}
```

```

/* 2. Set Dynamic Address from Static Address (SETDASA)
   Zephyr handles the 0x87 CCC and address mapping automatically
   based on the devicetree node provided. */
struct i3c_device_desc *target_desc = i3c_device_find(i3c_bus,
&I3C_DEVICE_ID(DT_NODELABEL(sensor_target)));

if (i3c_ccc_do_setdasa(target_desc) != 0) {
    LOG_ERR("SETDASA failed for target at 0x50");
    return;
}
LOG_INF("Dynamic Address assigned: 0x%02x", target_desc->dynamic_addr);

/* 3. Get FW Version via Private SDR Read */
uint8_t fw_reg = 0x40; // Example FW register
uint8_t fw_ver[3] = {0};

struct i3c_msg msgs[2];

/* Write the register address */
msgs[0].buf = &fw_reg;
msgs[0].len = 1;
msgs[0].flags = I3C_MSG_WRITE | I3C_MSG_STOP;

/* Read the 3-byte version */
msgs[1].buf = fw_ver;
msgs[1].len = 3;
msgs[1].flags = I3C_MSG_READ | I3C_MSG_STOP;

if (i3c_transfer(target_desc, msgs, 2) == 0) {
    LOG_INF("FW Version: %d.%d.%d", fw_ver[0], fw_ver[1], fw_ver[2]);
} else {
    LOG_ERR("Failed to read FW version");
}

}

```

Example 2:

```

#include <zephyr/kernel.h> #include <zephyr/drivers/i3c.h> #include <zephyr/logging/log.h>

LOG_MODULE_REGISTER(i3c_master, LOG_LEVEL_INF);

/* Get device pointers from Devicetree */ static const struct device *i3c_bus =
DEVICE_DT_GET(DT_NODELABEL(i3c0));

void main(void) { int ret; uint8_t fw_reg_addr = 0xFE; // Example: Vendor FW version register
uint8_t fw_version[3]; // To store Major, Minor, Patch

```

```

if (!device_is_ready(i3c_bus)) {
    LOG_ERR("I3C Bus not ready!");
    return;
}

/* 1. Find the target device descriptor in the bus list */
struct i3c_device_desc *target = i3c_device_find(i3c_bus,
&I3C_DEVICE_ID(DT_NODELABEL(my_sensor)));
if (!target) {
    LOG_ERR("Could not find target device in Devicetree");
    return;
}

/* 2. Execute SETDASA (CCC 0x87)
   This assigns 0x08 to the device at static address 0x50 */
ret = i3c_ccc_do_setdasa(target);
if (ret != 0) {
    LOG_ERR("SETDASA failed: %d", ret);
    return;
}
LOG_INF("SETDASA successful. Target dynamic address: 0x%02x", target-
>dynamic_addr);

/* 3. Prepare I3C Messages to read FW Version
   Message 0: Write the register address
   Message 1: Read the data bytes */
struct i3c_msg msgs[2];

msgs[0].buf = &fw_reg_addr;
msgs[0].len = 1;
msgs[0].flags = I3C_MSG_WRITE;

msgs[1].buf = fw_version;
msgs[1].len = sizeof(fw_version);
msgs[1].flags = I3C_MSG_READ | I3C_MSG_STOP;

/* 4. Perform the Private SDR Transfer */

ret = i3c_transfer(target, msgs, 2);
if (ret == 0) {
    LOG_INF("FW Version: v%d.%d.%d", fw_version[0], fw_version[1],
fw_version[2]);
} else {
    LOG_ERR("Failed to read FW version: %d", ret);
}

}
    
```

You may also refer to [Improved Inter-Integrated Circuit \(I3C\) Bus — Zephyr Project Documentation](#)

Himax Confidential
Do Not Copy

8. Security

WE2 Security Scheme

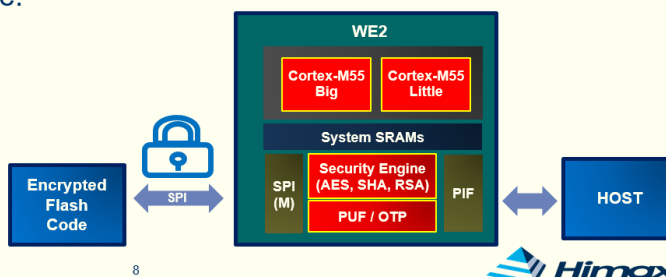
- WE2 embeds hardware security elements

- ❖ Cryptocell-CC312 engine

- AES128,
- SHA256
- RSA3072

- ❖ PUF / OTP security storage.

- generates chip unique keys
- stores Root RSA public key
- 128bits firmware version.



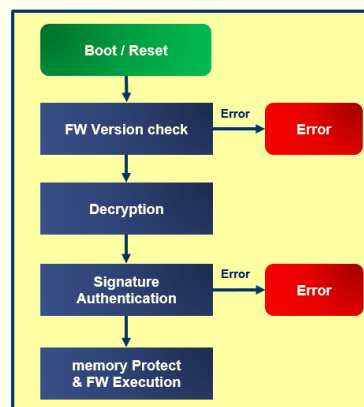
Himax Proprietary & Confidential



WE2 Secure Boot

- Secure boot will prevent loading malicious or unauthorized FW.

- ❖ Signature check with RSA3072
- ❖ Decryption with AES128
- ❖ Antirollback version control with 128bits OTP



Himax Proprietary & Confidential

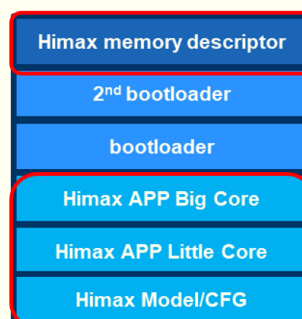
9



9. Firmware Update

WiseEye FW update – Fail safe mechanism

- WE2 Flash Image layout
 - ❖ Memory descriptor: describe flash memory layout
 - ❖ 2nd bootloader: executes OTA process
 - ❖ bootloader: load and verify APP from flash to SRAM
 - ❖ APP Big Core: the application of big core
 - ❖ APP Little Core: the application of little core if have
 - ❖ Model / CFG...: AI model or Application data
- Fail safe mechanism
 - ❖ OTA update process only update Himax APP Big core, little core, Model/CFG and memory descriptor.
 - ❖ The 2nd bootloader (OTA loader) and bootloader won't be updated
 - ❖ When update fail (e.g. power loss during update), WE2 will fail to verify the signatures of the images and load to 2nd bootloader (OTA loader) to wait for update again.



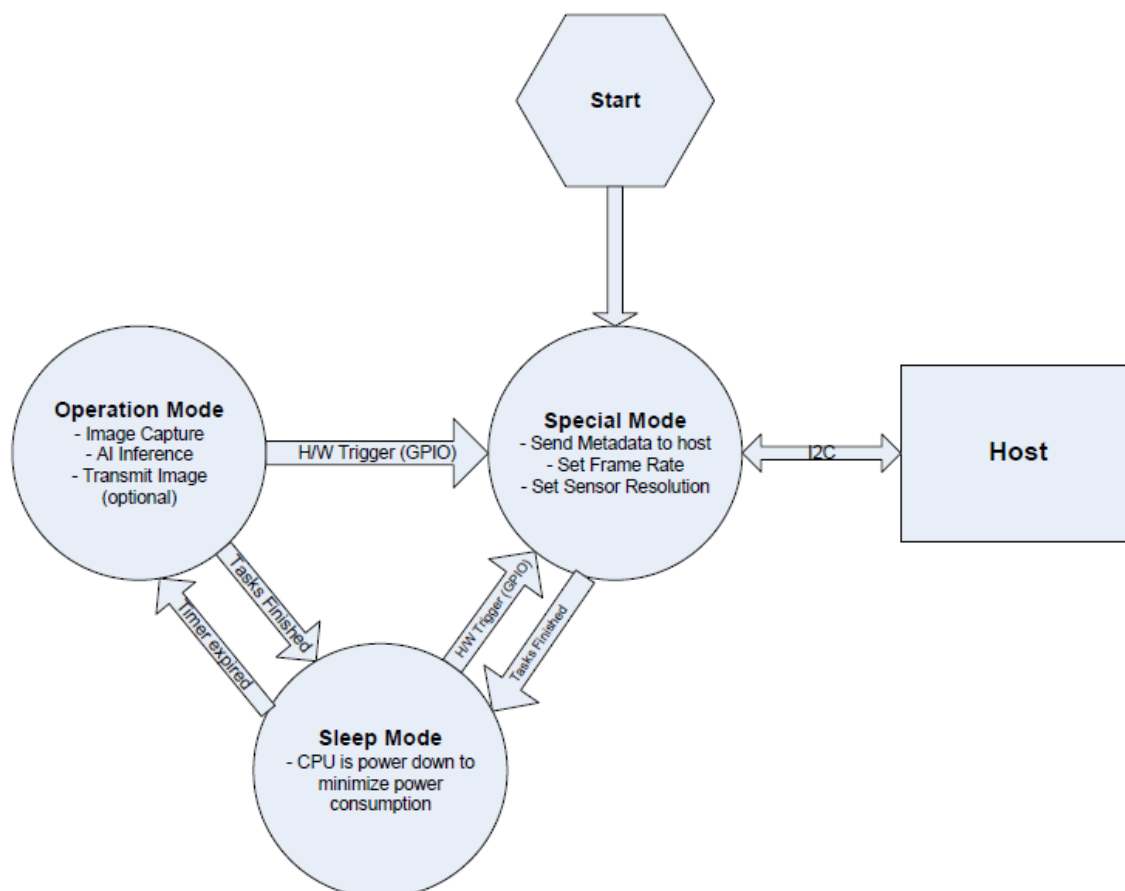
Himax Proprietary & Confidential

11

 **Himax**

10. State Machine

There are three states (modes) when WE2 application is running. Host controls transition of state machine by setting GPIO line and by sending I2C commands to WE2.



- **Operation Mode:** Image capture, AI inference, and other algorithms will be performed in this mode.

- **Sleep Mode:** When tasks in Operation Mode are complete, state-machine will enter this mode, where WE2 is in Power Down state to reduce power consumption. The internal timer will wake up CPU to enter Operation Mode.
- **Special Mode:** From either Operation Mode or Sleep Mode, when host sends interrupt through GPIO (PA0), state machine will enter Special Mode. In this mode, normal operations such as AI inference will be paused, and tasks of setting frame rate, or setting sensor resolution, or sending metadata to host, will be performed. The host communicates to WE2 by I2C/I3C to determine which special task to perform. The “Feature” byte in I2C/I3C command is to determine which special task to run.

Host Device(Master) sends write command	slv_addr (7bits)	R/W 1bit	Feature	cmd_type	Length	Data payload (N=Length) (MAX:256)				CRC (ecitt_crc16)
						cmd_param 1	cmd_param 2	...	cmd_param N	
I2C Format	SlaveID	0	wbuf[0]	wbuf[1]	wbuf[3:2]	wbuf[4]	wbuf[5]	...	wbuf[N+3]	wbuf[N+5:N+4]

- **Feature 0x82: Read metadata**
host sends: “82 00 00 00” to trigger WE2 responding with metadata.
host sends read command to retrieve response from WE2.

Below is the pseudocode:

```
GPIO_Write(GPIO_PORT3,1); /* Set PA0 high */
usleep(100000); /* Sleep 100ms */
i2c_cmd_write(0x82, 0x00, NULL, 0); /* Write for feature 0x82 */
usleep(10000); /* Sleep 10ms */
i2c_cmd_read(0x82, 0, i2c_response, 50); /* Read for feature 0x82 */
GPIO_Write(GPIO_PORT3,0); /* Set PA0 low */
```

- **Feature 0x83: Set sensor resolution**
host sends: “83 01 00 00” to set sensor resolution as QQVGA.
host sends: “83 02 00 00” to set sensor resolution as QVGA.

Below is the pseudocode to set sensor resolution to 320:

```
GPIO_Write(GPIO_PORT3,1);
usleep(20000);
i2c_cmd_write(0x83, RES_324x224, NULL, 0);
usleep(50000);
GPIO_Write(GPIO_PORT3,0);
```

- **Feature 0x84: Set frame rate**
host sends: “84 03 00 00” to set framerate as 3fps.
host sends: “84 05 00 00” to set framerate as 5fps.

Below is the pseudocode to set framerate as 3fps:

```
GPIO_Write(GPIO_PORT3,1);
usleep(20000);
i2c_cmd_write(0x84, FPS3, NULL, 0);
usleep(50000);
GPIO_Write(GPIO_PORT3,0);
```

State transition:

- By setting GPIO PA0 line high will transition from other states to state of 'Special Mode'.
- In states of 'Special Mode' or 'Operation Mode', when tasks are complete, transition to state of 'Sleep Mode'.
- An internal timer will transition from 'Sleep Mode' to 'Operation Mode'.

Note: The state machine should start with Special Mode. where frame rate and sensor resolution are configured. If the very first command is Read Metadata (Feature 0x82), you will read all FF in metadata because AI has not run yet.

11. Power Consumption Measurements

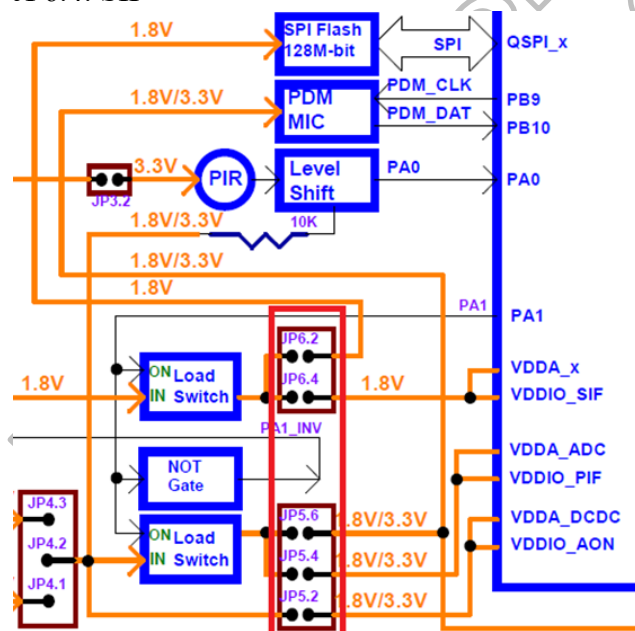
Test points JP5.2, JP5.4, JP6.2, and JP6.4 on circuit board for power measurements. These test points can be used to measure power for the following sources:

JP5.2: Core

JP5.4: PIF

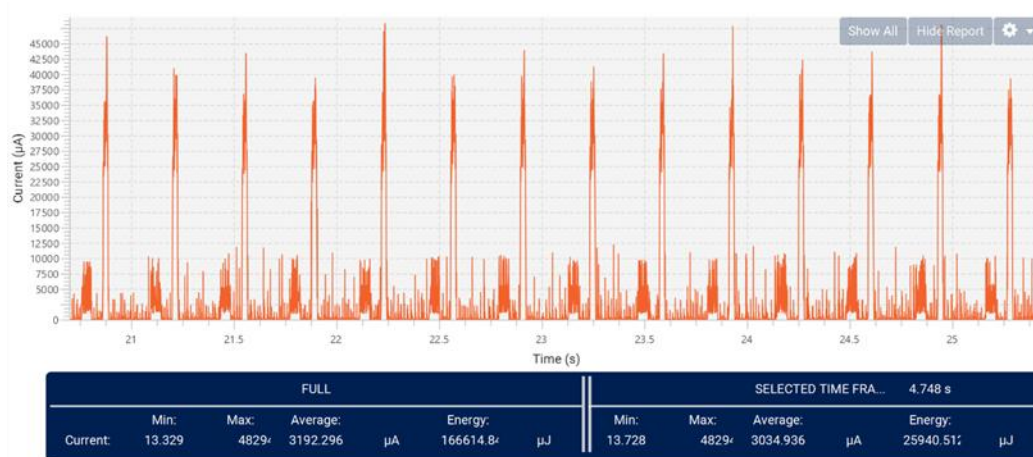
JP6.2: Flash

JP6.4: SIF

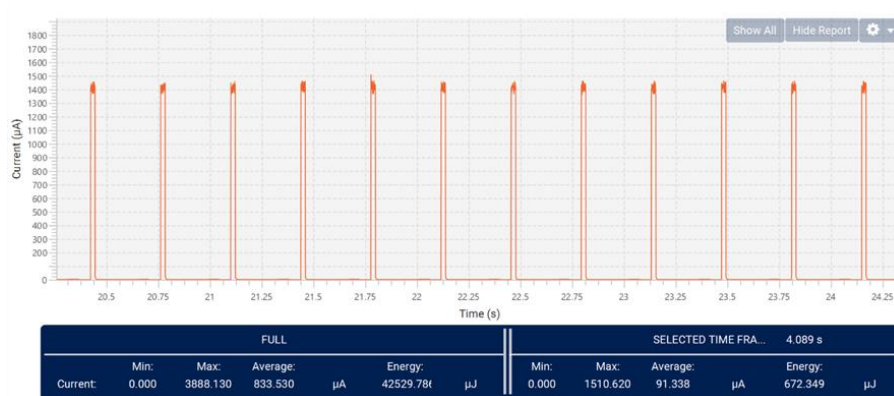


The following pictures are current measurements across the jumpers in scenario of operation mode, with settings of 3fps framerate and QVGA sensor resolution, with models Face Detection and Head Pose running.

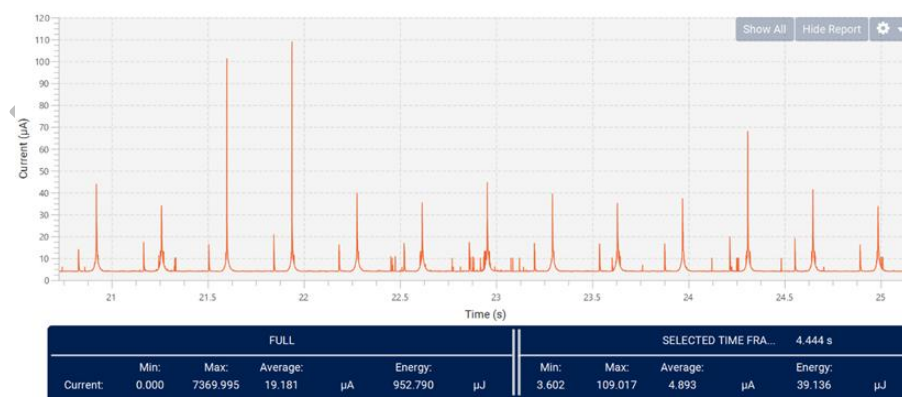
JP5.2: Core.



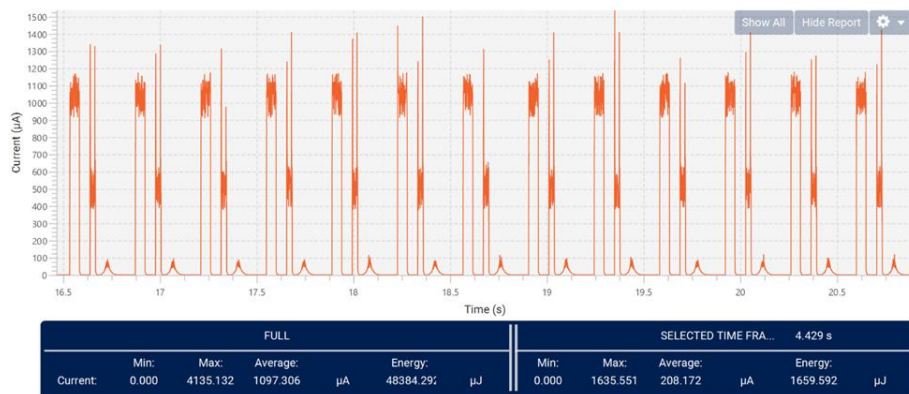
JP5.4: PIF



JP6.2: Flash



JP6.4: SIF



Total current draw is 3.34mA, and total power consumption is 6.00mW.